```
# Practical 01 --- Your MLOps Workbench

**A clean environment, a pinned library list, and a run that
repeats itself**

SCSE3040 Machine Learning Operations &middot; Bennett University
&middot; Session 2026-27

| | |
|---|---|
| **Lectures this follows** | L01-L02 |
| **Course Outcome** | CO1 |
| **Lab duration** | 120 minutes |
| **Memory needed** | about 250 MB (fine on a 4 GB or 8 GB
machine) |
| **Extra software** | nothing beyond the course venv |
| **Marks** | 10 |

---

## Aim

1. Find out which Python is actually running your code.
```

2. Write down the exact library versions your project needs.
3. Make a program that gives the same answer every single time.
4. Save your work in git with a proper first commit.

What you need to know first

MLOps (Machine Learning Operations) is the work of taking a model out of a notebook and keeping it running for real users. Almost every problem in that job comes from one sentence: "*but it works on my machine*".

It works on your machine because your machine has a particular Python, with particular libraries, at particular versions. Your friend's machine has different ones. The server has different ones again. The same code then gives a different answer, or no answer at all.

Professionals solve this in three steps, and today you will do all three.

1. **A virtual environment.** A private folder holding one Python and one set of libraries, used by one project only. Installing something for this course then cannot break another project on the same laptop.
2. **A pinned requirements file.** A plain text list saying *exactly* which version of each library you used --- `numpy==2.5.1`, not just `numpy`. Anybody can then rebuild your environment.

3. **A seed.** Machine learning uses random numbers: which rows go into training, where a model starts. Random means *different every run*, which means results you cannot check. Fixing the **seed** makes the randomness repeat, so your result can be checked by someone else.

Together these give you **reproducibility**: same code plus same data plus same versions gives the same answer, on any machine, on any day. Everything else in this course is built on top of that.

```
---

## Before you start

- The course virtual environment is installed. If not, follow
`labs/SETUP.md` first.
```

- You have opened this notebook from inside the `labs/P01-workbench/` folder.
- Nothing else. This is the first practical of the course.

How to run a cell

Click on a grey code cell, then press **Shift + Enter**. The cell runs and the cursor moves to the next one. A number appears in the `[ ]` on the left when the cell has finished.

Run the cells in order, from the top. A later cell almost always uses something an earlier cell created. If you jump ahead you will see a `NameError`, which just means "you have not made that thing yet".

If everything goes wrong, use the menu: **Kernel -> Restart Kernel and Clear All Outputs**, then start again from the first cell. Nothing is damaged by doing this.

In [3]:
```
# Step 0 --- check the workbench before we start.
# This cell only looks; it changes nothing. Run it and read the last line.

import sys
from pathlib import Path

print("Python  :", sys.version.split()[0])
print("Folder  :", Path.cwd().name)

_missing = []
for _name in ['numpy', 'pandas', 'sklearn']:
    try:
        __import__(_name)
    except ImportError:
        _missing.append(_name)

for _name in ['numpy', 'pandas', 'sklearn']:
    _mark = "missing" if _name in _missing else "ok"
    print(f"  {_name:<14} {_mark}")
```

```
if _missing:
    print()
    print("STOP. Some libraries are missing:", ", ".join(_missing))
    print("Ask your instructor to run the setup in labs/SETUP.md.")
else:
    print()
    print("All good. You can carry on to Step 1.")
```

```
Python  : 3.13.2
Folder  : P01-workbench
  numpy          ok
  pandas         ok
  sklearn        ok

All good. You can carry on to Step 1.
```

Step 0b --- the dataset

Every practical in this course uses the same 600 food deliveries. The next cell makes sure the file is there.

In [6]:
```
# The delivery-time dataset every practical in this course uses.
# If the file is missing we build it again from the same seed, so every
# student in the room gets byte-for-byte the same 600 rows.

import csv
from pathlib import Path

import numpy as np

SEED = 42
N_ROWS = 600
DATA = Path("..") / "data" / "delivery_times.csv"


def make_delivery_csv(path=DATA):
    """Write the 600-row delivery dataset. Same formula as the lectures."""
    rng = np.random.default_rng(SEED)
    distance_km = np.round(rng.uniform(0.5, 12.0, N_ROWS), 2)
    prep_time_min = np.round(rng.uniform(5, 30, N_ROWS), 0)
    traffic_level = rng.integers(1, 4, N_ROWS)
    rain = rng.binomial(1, 0.25, N_ROWS)
    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["distance_km", "prep_time_min", "traffic_level",
```

```
                    "rain", "delivery_min"])
        for i in range(N_ROWS):
            w.writerow([distance_km[i], int(prep_time_min[i]),
                        int(traffic_level[i]), int(rain[i]), delivery_min[i]])
    return path


if not DATA.exists():
    make_delivery_csv()
    print("dataset rebuilt ->", DATA)
else:
    print("dataset found   ->", DATA)
```

```
dataset found   -> ..\data\delivery_times.csv
```

Walkthrough

Read each step, then run its cell.

Step 1 --- Which Python is running this notebook?

Your laptop probably has more than one Python. There is the one Windows ships, maybe one from Anaconda, and the one this course installed. The very first thing to check is which one you are actually using, because that decides which libraries you can see.

In [7]:
```
import sys
from pathlib import Path

print("Python version :", sys.version.split()[0])
print("Python program :", sys.executable)

# A virtual environment is just a folder. If the path above sits
# inside a folder called .venv, you are in the course environment.
in_venv = ".venv" in sys.executable.replace("\\", "/")
print("Inside .venv    :", in_venv)
print("Working folder  :", Path.cwd())
```

```
Python version : 3.13.2
Python program : C:\Users\ishak\Documents\MLOps\.venv\Scripts\python.exe
Inside .venv    : True
Working folder  : C:\Users\ishak\Documents\MLOps\P01-workbench
```

If `Inside .venv` says `False`, you are running the wrong Python. Tell your instructor before going further --- later cells will fail in confusing ways.

Step 2 --- What is installed in this environment?

A library is code somebody else wrote that you can use. Your project depends on several. Each has a **version number**, and versions matter: `scikit-learn` 1.9 does not behave exactly like 1.2.

Let us ask Python what it has.

In [8]:
```
from importlib.metadata import version

LIBRARIES = ["numpy", "pandas", "scikit-learn", "matplotlib"]

for name in LIBRARIES:
    print(f"{name:<15} {version(name)}")
```

```
numpy           2.5.1
pandas          2.3.3
scikit-learn    1.9.0
matplotlib      3.11.1
```

Write these numbers down somewhere. In six months, when your code suddenly stops working, the first question anyone will ask is *"which versions were you using?"*

Step 3 --- Freeze those versions into requirements.txt

`requirements.txt` is the standard file name for that list. One library per line. The `==` means *exactly this version*, which is what "pinned" means.

We will write it into a folder called `work/`, which is where everything you create today will live.

In [9]:
```
WORK = Path("work")
WORK.mkdir(exist_ok=True)

lines = [f"{name}=={version(name)}" for name in LIBRARIES]
(WORK / "requirements.txt").write_text("\n".join(lines) + "\n",
                                       encoding="utf-8")

print("wrote", WORK / "requirements.txt")
print("-" * 40)
print((WORK / "requirements.txt").read_text(encoding="utf-8"))
```

```
wrote work\requirements.txt
----------------------------------------
numpy==2.5.1
pandas==2.3.3
scikit-learn==1.9.0
matplotlib==3.11.1
```

On a new machine, one command rebuilds your whole environment from this file:

```
pip install -r requirements.txt
```

That single line is why "works on my machine" stops being an excuse.

Step 4 --- Random numbers change every time you ask

Now the third idea: randomness. Run the next cell, then run it a second time. Look at the numbers.

In [10]:
```
import numpy as np
```

```
careless = np.random.default_rng()   # no seed given
print("three random numbers:", np.round(careless.uniform(0, 10, 3), 2))
```

```
three random numbers: [0.56 5.6  5.9 ]
```

Different every time. That is fine for a dice game and a disaster for a model: your accuracy score would change every run and nobody could tell whether your improvement was real.

Step 5 --- A seed makes the randomness repeat

A **seed** is a starting number for the random number generator. Same seed in, same sequence out --- on any machine, forever.

`42` is the number everyone uses, for a joke that is older than you.

In [11]:
```
first  = np.random.default_rng(42).uniform(0, 10, 3)
second = np.random.default_rng(42).uniform(0, 10, 3)

print("first run :", np.round(first, 2))
print("second run:", np.round(second, 2))
print("identical :", np.array_equal(first, second))
```

```
first run : [7.74 4.39 8.59]
second run: [7.74 4.39 8.59]
identical : True
```

This is the single most important habit in this course. **Every time you use randomness, set a seed.**

Step 6 --- Build the delivery dataset, twice

Now we use the seed for something real. The course project predicts how many minutes a food delivery will take. The dataset has 600 orders and five columns.

We will build it twice and prove the two files are identical, using a **checksum** --- a short fingerprint of a file's contents. If one single character differs, the fingerprint changes completely.

In [12]:
```
import hashlib

def sha256_of(path):
    """A short fingerprint of a file's exact contents."""
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()

make_delivery_csv(WORK / "run_a.csv")
make_delivery_csv(WORK / "run_b.csv")

fp_a = sha256_of(WORK / "run_a.csv")
fp_b = sha256_of(WORK / "run_b.csv")

print("run A:", fp_a[:16], "...")
print("run B:", fp_b[:16], "...")
print("identical files:", fp_a == fp_b)
```

```
run A: 9e9f7a46c817d5bb ...
run B: 9e9f7a46c817d5bb ...
identical files: True
```

Two separate runs, byte-for-byte the same file. That is reproducibility you can actually prove to somebody.

Step 7 --- Look at the data you just made

CSV (Comma Separated Values) is a plain text table. `pandas` is the library that reads one into something you can work with.

In [13]:
```
import pandas as pd

orders = pd.read_csv(DATA)

print("rows, columns:", orders.shape)
print()
print(orders.head())
print()
print(orders.describe().round(1))
```

```
rows, columns: (600, 5)

   distance_km  prep_time_min  traffic_level  rain  delivery_min
0         9.40             17              1     0          51.3
1         5.55             24              2     1          54.2
2        10.37             28              3     0          67.7
3         8.52             23              2     0          51.2
4         1.58             29              2     1          42.1

       distance_km  prep_time_min  traffic_level   rain  delivery_min
count        600.0          600.0          600.0  600.0         600.0
mean           6.2           17.6            2.0    0.3          46.6
std            3.3            7.4            0.8    0.4          12.1
min            0.6            5.0            1.0    0.0          18.5
25%            3.2           11.0            1.0    0.0          37.8
50%            6.2           17.0            2.0    0.0          46.5
75%            9.1           24.0            3.0    1.0          55.7
max           12.0           30.0            3.0    1.0          75.1
```

`distance_km` is how far the rider goes, `prep_time_min` is how long the restaurant takes, `traffic_level` is 1 to 3, `rain` is 0 or 1, and `delivery_min` is the answer we will eventually try to predict.

Step 8 --- Record what you ran

The last habit: write down the facts of the run itself. Months later this file is the only thing that can tell you what produced a result.

In [14]:
```
import json

run_info = {
    "python": sys.version.split()[0],
    "seed": SEED,
    "rows": len(orders),
```

```
    "data_sha256": sha256_of(DATA),
    "libraries": {n: version(n) for n in LIBRARIES},
}

(WORK / "run_info.json").write_text(json.dumps(run_info, indent=2),
                                    encoding="utf-8")
print(json.dumps(run_info, indent=2))
```


```
{
  "python": "3.13.2",
  "seed": 42,
  "rows": 600,
  "data_sha256": "9e9f7a46c817d5bb81c3e458f66617d17a116a3218c6bb5f555c05dd831f59a
1",
  "libraries": {
    "numpy": "2.5.1",
    "pandas": "2.3.3",
    "scikit-learn": "1.9.0",
    "matplotlib": "3.11.1"
  }
}
```


Step 9 --- Save your work in git

git is the tool that remembers every version of your files. A **commit** is one saved point you can always come back to.

We will make a small repository inside `work/` so nothing else on your machine is touched. Read the output: git prints what it did.

In [15]:
```
import subprocess

def git(*args):
    """Run one git command inside work/ and show what it said."""
    done = subprocess.run(["git", *args], cwd=WORK,
                          capture_output=True, text=True)
    print("$ git", " ".join(args))
    print((done.stdout + done.stderr).strip() or "(no output)")
    print("-" * 50)
    return done

if not (WORK / ".git").exists():
    git("init", "-q")
git("config", "user.name", "SCSE3040 Student")
git("config", "user.email", "student@bennett.edu.in")
git("add", "requirements.txt", "run_info.json")
git("commit", "-q", "-m", "P01: pinned requirements and run record")
git("log", "--oneline")
```

```
$ git config user.name SCSE3040 Student
(no output)
--------------------------------------------------
$ git config user.email student@bennett.edu.in
(no output)
--------------------------------------------------
$ git add requirements.txt run_info.json
(no output)
--------------------------------------------------
$ git commit -q -m P01: pinned requirements and run record
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        my_requirements.txt
        run_a.csv
        run_b.csv

nothing added to commit but untracked files present (use "git add" to track)
--------------------------------------------------
$ git log --oneline
13a3be1 P01: pinned requirements and run record
--------------------------------------------------
```

Out[15]:
```
CompletedProcess(args=['git', 'log', '--oneline'], returncode=0, stdout='13a3be
1 P01: pinned requirements and run record\n', stderr='')
```

You now have the four habits every later practical assumes: the right environment, pinned versions, a fixed seed, and a commit. Everything from Docker to Kubernetes is only useful once these are in place.

Your turn

The walkthrough above is finished. Now you write some code.

There are **3 tasks**. Each one is small. Each one has a hint. Do them in order.

Where you see `# TODO`, replace that line with your own code. Do not delete the variable name on the left of the `=` sign --- the self-check at the bottom looks for exactly that name.

When you have tried all three, run the **self-check** cell at the end. It prints a table telling you which tasks are correct. You can run it as many times as you like.

Task T1 --- Change the seed

```
The dataset was built with seed **42**. Build it with seed **7**
instead, and report the first three `distance_km` values.

Do not change `make_delivery_csv`. Use
`np.random.default_rng(7)`
directly, exactly the way the function does: draw 600 numbers
between
`0.5` and `12.0`, round them to 2 decimal places, then take the
```

first
three.

Put your answer in a list called `T1_first_three`.

> **Hint.** One line does it: `np.round(np.random.default_rng(7).uniform(0.5, 12.0, 600), 2)[:3]`. Wrap it in `list(...)` so it becomes a plain Python list.

In [14]:
```
# TODO: replace None with your one-line answer.
T1_first_three = list(np.round(np.random.default_rng(7).uniform(0.5, 12.0, 600),

print("T1_first_three =", T1_first_three)
```

```
T1_first_three = [np.float64(7.69), np.float64(10.82), np.float64(9.42)]
```

Task T2 --- Write your own pinned requirements file

Write a file `work/my_requirements.txt` containing exactly three lines,
one each for `numpy`, `pandas` and `scikit-learn`, each pinned to the
version actually installed on this machine.

Do not type the version numbers by hand. Ask Python for them, so the
file is right on any machine.

> **Hint.** You already saw the pattern in Step 3. `version("numpy")` gives the number; an f-string like `f"numpy=={version('numpy')}"` gives the line. Join three of them with `"\n"` and use `(WORK / "my_requirements.txt").write_text(...)`.

In [17]:
```
MY_LIBS = ["numpy", "pandas", "scikit-learn"]

# TODO: build one "name==version" line per library, then write them
#       to work/my_requirements.txt with a newline between them.
my_lines = [f"{name}=={version(name)}" for name in MY_LIBS]
(WORK / "my_requirements.txt").write_text("\n".join(lines) + "\n",
                                          encoding="utf-8")

print(my_lines)
print((WORK / "my_requirements.txt").read_text(encoding="utf-8"))
```

```
['numpy==2.5.1', 'pandas==2.3.3', 'scikit-learn==1.9.0']
numpy==2.5.1
pandas==2.3.3
scikit-learn==1.9.0
matplotlib==3.11.1
```

Task T3 --- Fingerprint a run

Write a function `fingerprint(path)` that takes the path of a CSV file
and returns a **dictionary** with exactly these three keys:

- `"rows"` --- how many data rows the file has, not counting the header
- `"sha256"` --- the checksum of the file, using the `sha256_of` helper
  from Step 6
- `"seed"` --- the seed the course uses, which is in the variable `SEED`

Then call it on `DATA` and store the result in `T3_fp`.

> **Hint.** `pd.read_csv(path)` gives you a table; `len(...)` on it gives the row count without the header. `sha256_of(path)` is already written for you. Return them as `{"rows": ..., "sha256": ..., "seed": ...}`.

In [22]:
```
def fingerprint(path):
    # TODO: return a dictionary with the keys rows, sha256 and seed.
     return {
        "rows": len(pd.read_csv(path)),
        "sha256": sha256_of(path),
        "seed": SEED
    }


T3_fp = fingerprint(DATA)
print(T3_fp)
```

```
{'rows': 600, 'sha256': '9e9f7a46c817d5bb81c3e458f66617d17a116a3218c6bb5f555c05dd831f59a1', 'seed': 42}
```

Self-check

Run the cell below to mark your work.

In [23]:
```
# ---------------------------------------------------------------------
# SELF-CHECK --- run this when you have attempted the tasks above.
# It never breaks your notebook. A task you have not done yet simply
# shows FAIL.
# ---------------------------------------------------------------------

_results = []


def _check(label, fn):
    """Evaluate one graded condition without ever raising."""
    try:
        ok = bool(fn())
    except Exception:
        ok = False
    _results.append((label, ok))
```

```
_check('T1 | T1_first_three holds three numbers', lambda: len(T1_first_three) ==
_check('T1 | those are the first three distances for seed 7', lambda: all(abs(fl
_check('T2 | work/my_requirements.txt exists', lambda: (WORK / 'my_requirements.
_check('T2 | it pins all three libraries with ==', lambda: sum(1 for ln in (WORK
_check('T3 | fingerprint() returns the three required keys', lambda: set(T3_fp)
_check('T3 | it counts 600 data rows and uses seed 42', lambda: T3_fp['rows'] ==
_check('T3 | the checksum matches the file on disk', lambda: T3_fp['sha256'] ==

print("======================================================================")
print("SELF-CHECK   Practical 01 --- Your MLOps Workbench")
print("======================================================================")
for _label, _ok in _results:
    print(f"  [{'PASS' if _ok else 'FAIL'}]  {_label}")
print("----------------------------------------------------------------------")
_passed = sum(1 for _, _ok in _results if _ok)
print(f"  {_passed} of {len(_results)} checks passed")
print("======================================================================")
if _passed == len(_results):
    print("Well done. Save the notebook and submit it.")
else:
    print("Read the FAIL lines above, fix those tasks, run this cell again.")
```

```
======================================================================
SELF-CHECK   Practical 01 --- Your MLOps Workbench
======================================================================
  [PASS]  T1 | T1_first_three holds three numbers
  [PASS]  T1 | those are the first three distances for seed 7
  [PASS]  T2 | work/my_requirements.txt exists
  [PASS]  T2 | it pins all three libraries with ==
  [PASS]  T3 | fingerprint() returns the three required keys
  [PASS]  T3 | it counts 600 data rows and uses seed 42
  [PASS]  T3 | the checksum matches the file on disk
----------------------------------------------------------------------
  7 of 7 checks passed
======================================================================
Well done. Save the notebook and submit it.
```

What to submit

1. This notebook, with every cell run and its output visible.
2. The file `work/my_requirements.txt` that you wrote in Task T2.
3. A screenshot of the output of the last walkthrough cell (`git log`).

Name your file `PXX_<your-roll-number>.ipynb` before you upload it.

How this practical is marked

| What is marked | Marks | |---|---| | Walkthrough run end to end, outputs visible | 3 | | Task T1 --- seeds control randomness | 2 | | Task T2 --- a correctly pinned requirements file | 2 | | Task T3 --- a working fingerprint function | 3 | | **Total** | **10** |

Read more

- Python docs --- Virtual environments and packages --- https://docs.python.org/3/tutorial/venv.html

- pip --- Requirements files --- https://pip.pypa.io/en/stable/reference/requirements-file-format/
- NumPy --- Random generator and seeds --- https://numpy.org/doc/stable/reference/random/generator.html
- Pro Git --- Getting started --- https://git-scm.com/book/en/v2/Getting-Started-First-Time-Git-Setup